

Module 6: Component Communication & Content Projection

Building Reusable and Interactive Components

ANGULAR ENTERPRISE SERIES

Module Overview

By the end of this module, you will be able to:

01

Understand Communication Patterns

Recognize when and why components need to exchange data and events.

02

Exchange Data Between Components

Implement parent-to-child and child-to-parent communication effectively.

03

Share State Across Components

Use shared services and signals to maintain consistent application state.

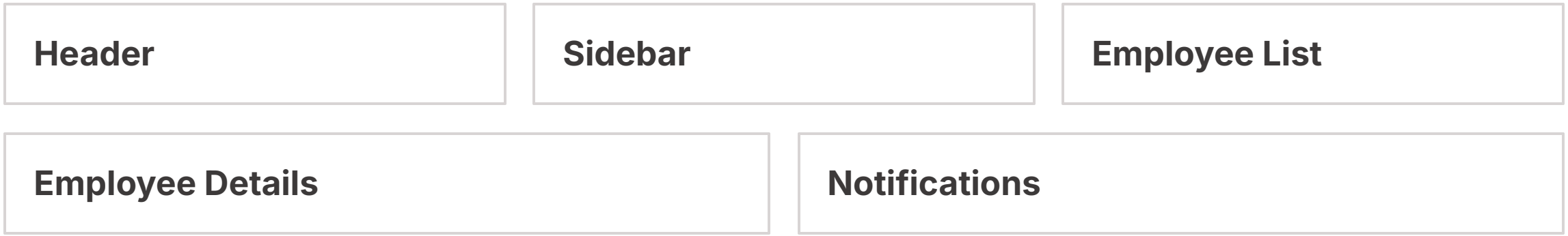
04

Build With Content Projection

Create flexible, reusable components using single and multi-slot projection.

Why Component Communication?

Enterprise applications are composed of many independent components that must coordinate data and events without becoming tightly coupled.



Key Point: Effective communication enables modular, maintainable, and scalable applications — without sacrificing component independence.



DATA FLOW

Parent-to-Child Communication

The parent component owns and provides data — the child simply consumes it. This creates a clear, predictable top-down data flow.

Employee Details

Pass selected employee record from list to detail panel.

Dashboard Widgets

Supply configuration and data from the parent dashboard.

User Profile

Deliver logged-in user context to any consuming child.

EVENT FLOW

Child-to-Parent Communication

Children notify parents when actions occur — keeping UI components decoupled from business logic and easy to reuse across contexts.

Result: Event-driven architecture with loose coupling and better reusability.



Save / Delete

Form actions bubble up to the managing parent.

Update / Cancel

Edit workflows emit events for parent to handle state.

Form Submission

Collected form data is emitted to the parent for processing.

Shared State Communication

When multiple unrelated components need access to the same data, a shared service or signal acts as a single source of truth.



Single Source of Truth

Consistent State

No Duplicate Logic

Shared services eliminate the need to pass data through multiple layers of unrelated components — keeping each component focused on its own responsibility.

Choosing the Right Communication Pattern

Selecting the correct pattern from the start prevents architectural debt. Match the mechanism to the actual relationship between components.

Scenario	Recommended Approach
Parent needs to configure or supply data to a child	**Parent → Child** (Input binding)
Child needs to notify the parent of a user action	**Child → Parent** (Output / EventEmitter)
Multiple sibling or unrelated components share data	**Shared Service / Signals**

Best Practice: Choose the simplest communication mechanism that meets the business requirement — don't over-engineer.

Content Projection

Content Projection lets a component define *where* externally provided content appears — making it a reusable container without hardcoding its inner structure.

Reusable Layouts

Define a shell once and project different content into it everywhere it's used.

No Duplicate Markup

Avoid copy-pasting structural HTML across similar components like cards and dialogs.

Maximum Flexibility

Consumers control the content; the component controls the layout and style.



SINGLE SLOT

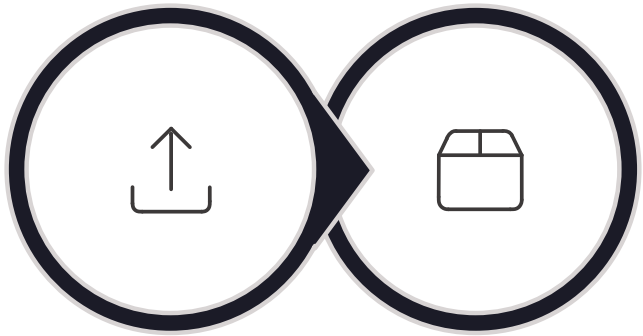
Single Slot Projection

A single `ng-content` placeholder accepts all content provided by the parent. The component wraps it with consistent styling and structure.

Common use cases:

- Card and Panel containers
- Alert and notification boxes
- Generic wrapper components
- Collapsible accordion sections

The consuming parent decides what goes inside; the component handles borders, shadows, padding, and layout.



Parent Provides **Component Renders**

Multi Slot Projection

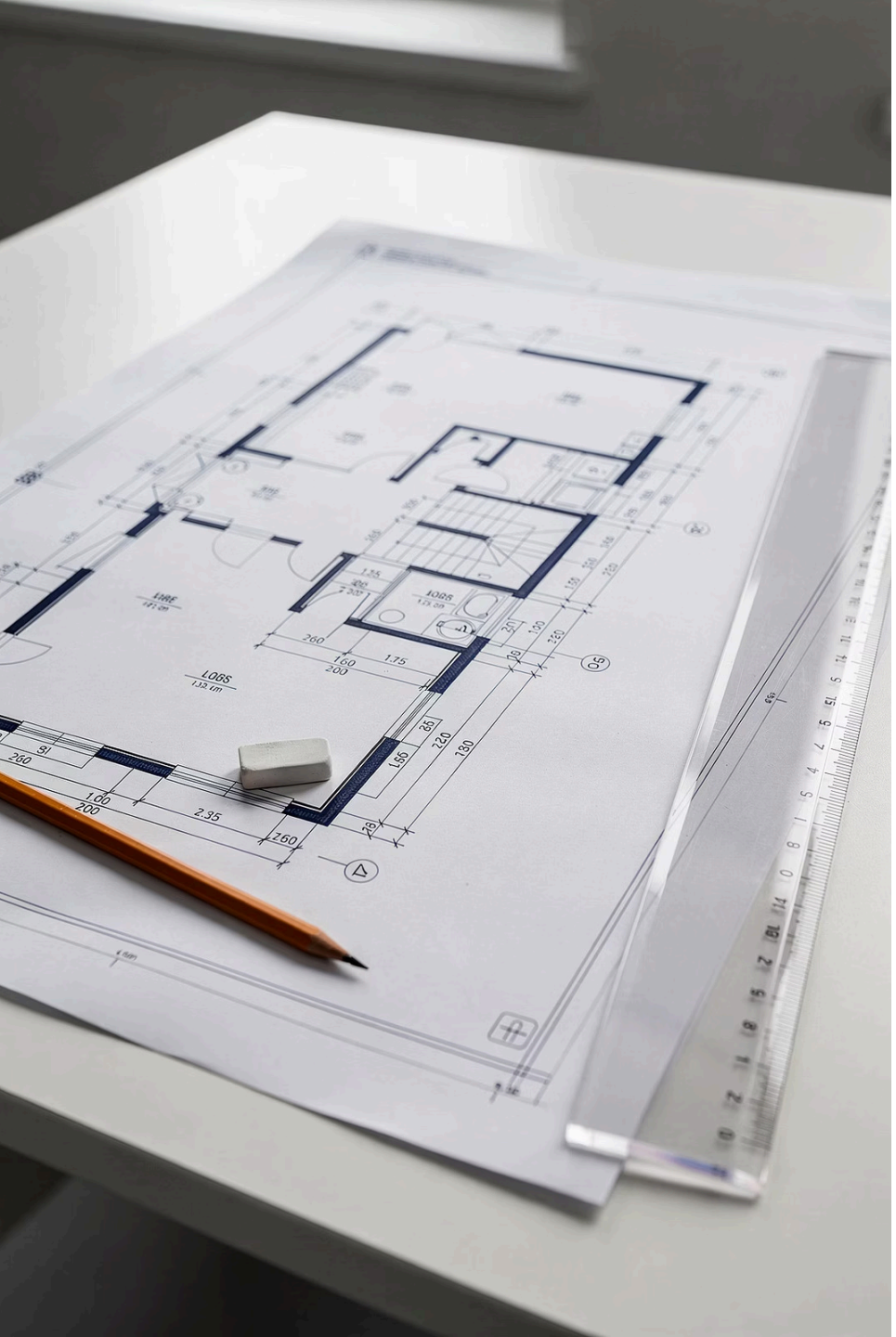
Named slots let different sections of the parent's template map to distinct regions inside the component — enabling rich, structured layouts without duplication.



Enterprise Use Cases

- **Dashboard Cards** — custom titles, data, and action buttons per card
- **Dialog Windows** — header, scrollable body, and footer actions
- **Report Panels** — label, chart content, and export controls
- **Employee Detail Views** — profile header, detail body, edit footer

Multi-slot projection is the foundation of truly composable, enterprise-grade UI libraries.



Communication Best Practices

Parent Owns the Data

State lives in the parent or a shared service — children receive it, they don't invent it.

Children Raise Events

UI components emit events and let the parent decide how to respond.

Single Responsibility

Each component does one thing well — presentation or logic, not both.

Build Reusable Presenters

Keep presentation components pure — no HTTP calls, no side effects.

Communication Anti-Patterns

These patterns are common sources of bugs, poor performance, and long-term maintenance pain in enterprise applications.

🚫 Deep Chains

Passing data through 4+ component layers creates fragile, hard-to-refactor code.

🚫 Tight Coupling

Components that reference each other directly cannot be reused or tested in isolation.

🚫 Duplicate State

Maintaining the same data in two places leads to inconsistency and hard-to-track bugs.

🚫 Business Logic in UI

Embedding API calls or domain rules inside components breaks separation of concerns.

⊗ These practices make applications difficult to maintain, debug, and scale as team size and feature complexity grow.

Module Summary

What We Covered

- [ ] **Parent-to-Child & Child-to-Parent Communication**
- [ ] **Shared State with Services & Signals**
- [ ] **Single & Multi Slot Content Projection**
- [ ] **Best Practices & Anti-Patterns**

Next Module

Dependency Injection & Services

- How Angular's DI system works
- Creating and registering services
- Provider configuration strategies
- Understanding the injector hierarchy